

Solution specifications what to build, how well, and where to stop

Before a single screen gets designed, an innovator writes the spec : the functional and non-functional requirements , the constraints they must work inside, and the scope that draws the line around the whole thing. Get this right and the build almost designs itself. Get it vague and the project drowns.

What this key knowledge covers

This summary distils everything you need for **KK05 — Solution specifications what to build, how well, and where to stop** in Innovative Solutions. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

The dot point, decoded

The VCAA study design lists this key knowledge in one breath:

That short line hides four jobs. A **specification** (or "spec") is the agreed description of what an innovative solution must be. It is produced in the **Analysis stage** of the problem-solving methodology — the stage that answers "*what exactly are we building, and inside what limits?*" — and it becomes the **yardstick you evaluate against** at the very end.

Almost every U201 SAC and exam question about an innovative solution touches the spec. The marking rubric explicitly rewards students who **document detailed functional and non-functional requirements, scope and constraints** — not students who just list a couple. The difference between a C and an A here is precision: specific, testable statements tied to the scenario.

One spec, four questions

The cleanest way to never mix these up: each part of the spec answers exactly **one question**. Memorise the questions, and classification becomes almost automatic.

Things the solution **does**. Features and behaviours you could click and demonstrate.

The **qualities** of how it performs — fast, secure, usable, reliable, accessible.

Restrictions **imposed from outside** — budget, deadline, available tech, the law.

What the solution **will and won't** include — the line drawn around the work.

Functional = **WHAT** it does. Non-functional = **HOW WELL** it does it. Constraint = a **limit pushed on you**. Scope = a **line you draw**. Keep those four questions in your head and the classification questions almost answer themselves.

Meet SwapShelf

Priya is in Year 11. For her Innovative Solutions outcome, her team of four noticed a real problem: every year, students throw out usable textbooks, calculators, uniforms and lab coats — while the year below pays full price for the same gear. It's wasteful and expensive. That's a genuine *problem, need and opportunity*, and it ties straight to **UN Sustainable Development Goal 12** (responsible consumption).

Their idea: **SwapShelf** — a web app where students at their school list second-hand school gear they no longer need, browse what others have posted, and message to arrange a free swap or hand-over. No money changes hands.

SwapShelf is perfect for this topic because it forces every part of the spec to do real work: it has obvious features, real quality stakes (student data, slow Wi-Fi), hard limits (no money, one term), and a boundary that has to be drawn carefully or the team will never finish.

Functional requirements

The easy test: *could you demonstrate it by clicking?* If yes, it's functional. Functional requirements are usually written as "**The solution must / will [verb] ...**" — and the verb is the giveaway: *create, list, search, send, calculate, display, delete, log in*.

SwapShelf's functional requirements

- The solution **must let a student create a listing** with a photo, title, category and condition.
 - The solution **must let users browse and search/filter** listings by category.
 - The solution **must let a user message** another student to arrange a swap.
 - The solution **must let a student mark an item as "swapped"**, removing it from active listings.
 - The solution **must require students to log in** with their school account.
- "The app should be good and help students swap things" is **not** a functional requirement — it's a wish. A real one names a concrete behaviour: "*must let a student create a listing with a photo*." Examiners reward the specific, testable version every time.

Non-functional requirements

Non-functional requirements don't add features; they set the **quality bar** for the features you already have. They often hide inside adjectives and adverbs — *fast, secure, easy, reliable, accessible* — and the strong ones attach a **measurable target** so you can actually test whether you hit them. People nickname them the **"-ilities."**

SwapShelf's non-functional requirements

- **Usability** — a new student can post a listing within 2 minutes, with no instructions.
- **Performance** — search results load in under 2 seconds on the school Wi-Fi.
- **Security** — only logged-in students can see listings; messages stay private between the two parties.

- **Accessibility** — text meets WCAG AA contrast and works with a screen reader.
- **Reliability** — available across school hours with under 1% downtime.

"Let a student post a listing" is the **function** (what). "...within 2 minutes, on a cheap phone, over slow Wi-Fi" is the **non-functional** requirement (how well). The function can work perfectly and still *fail* the non-functional bar if it takes 30 seconds and locks up. That's the whole distinction in one sentence.

Sort it out: functional vs non-functional

Eight SwapShelf statements. Drag each card into the bin you think it belongs in — or tap a card, then tap a bin. You'll get instant feedback and a running score. Ask yourself the test every time: *what*, or *how well*?

Constraints

The key word is **imposed**. You don't choose a constraint; it's handed to you by the budget, the calendar, the available hardware, the school's rules or the law. Constraints usually fall into a few familiar types:

"Must comply with the Privacy Act" is a **constraint** (a legal limit imposed on you), not a functional requirement. The *requirement* it produces might be "messages stay private" (non-functional). Ask: *is this a limit pushed onto the project, or a thing the solution does / a quality it has?* Limit → constraint. Does / quality → requirement.

Match the constraint to its type

Pick the type for each SwapShelf limit. Get them all green.

Scope & the Scope Box

A good scope statement always names **both sides** of the line. Listing what's in is only half the job; naming what's *out* is what stops the project from quietly growing until it can't be finished — a problem called **scope creep**.

Notice the trade-off the box makes physical: every feature you pull *inside* the boundary costs time and effort. With only one term and \$0, the team can't fit everything — so they draw the scope **small and deliberate**, keeping the core swap loop in and pushing payments, delivery and native apps out. **Scope is the lever that makes the requirements achievable within the constraints.**

The whole spec, on one page

Here's how the four parts read as a single specification — the document Priya's team would hand to whoever builds SwapShelf. Each part has a clear job, and together they leave no room for "wait, what were we building again?"

This is the part students forget: the spec you write in **Analysis** comes back in **Evaluation**. You can only judge whether SwapShelf "worked" by checking it against the requirements you set. A vague spec means a vague evaluation — and lost marks at *both* ends of the project. Write it sharp once, use it twice.

Command-word decoder

The verb in the question tells you how much to write and what shape your answer takes. Tap to expand.

Build an answer with P-E-L

Most spec questions want a tight three-move answer: state the **point** using the correct term, back it with an **example** from the scenario, then **link** it to the question. Write a response below and watch the trainer detect each move live.

Point: This is a non-functional requirement. **Example:** "search in under 2 seconds" doesn't add a feature — searching is the function; the 2-second target sets how *fast* it must be. **Link:** Because it describes the quality (speed) of a function rather than a new behaviour, it is non-functional, with a measurable target that makes it testable.

VCE-style practice questions

Try each one on paper **first**, then reveal the weak answer (and why it loses marks), the strong answer, and the marking guide. Mark yourself honestly — your score fills the dock in the corner and saves automatically.

- 1 mark — functional defined as a feature/behaviour (what it does).
- 1 mark — non-functional defined as a quality of performance (how well). An example each strengthens but the marks sit on the definitions.
- 1 mark — a valid functional requirement identified.
- 1 mark — justified (a behaviour/feature the system performs).
- 1 mark — a valid non-functional requirement identified.
- 1 mark — justified (a quality of performance; measurable target rewarded).
- 1 mark — constraint = an externally imposed limit/restriction.
- 1 mark — requirement = what the solution must do, or a quality it must have.
- 1 mark — a correct SwapShelf example that illustrates the difference.
- 1 mark — a valid in-scope element.
- 1 mark — a valid out-of-scope element.
- 1 mark — scope explained as a boundary that prevents scope creep.
- 1 mark — links defining scope to meeting the constraints (time/budget/team size).
- 1-2 marks — identifies the relevant constraints (economic / technical / time).
- 1-2 marks — explains the impact on requirements, incl. realistic non-functional targets.
- 1-2 marks — explains the impact on scope (what's cut vs kept) with reasoning.
- Top band — coherent, scenario-anchored analysis showing scope set to fit the constraints.

One-screen recap

- Answers: **what does it do?**
- A feature / behaviour you could click
- "The solution must [verb]..."
- SwapShelf: **create a listing**
- Answers: **how well?**
- A quality: speed, security, usability...
- Best with a **measurable target**
- SwapShelf: **search in <2 s**
- A **limit imposed** from outside

- Economic · technical · time · legal · social
- Not chosen — handed to you
- SwapShelf: **\$0 · one term**
- The **boundary**: in vs out
- Always name what's **excluded**
- Stops **scope creep**
- Set to fit the constraints
- The spec lives in **Analysis**
- Requirements = what + how well
- Constraints squeeze; scope adjusts
- Reused as the **evaluation yardstick**
- Call speed/security "functional"
- Write vague, untestable wishes
- List only in-scope, forget out
- Confuse a constraint with scope

U2·O1 KK05 · Solution specifications · Next up: KK06 — Collecting user needs

Key terms

Term	Meaning
Economic / financial money available	\$0 budget — the team must use only free hosting and free tools.
Technical hardware, software, platform	The school blocks app installs, so SwapShelf must be a <i>web</i> app that runs in a browser on students' existing phones.
Time the deadline	It must be finished and tested within one term (about 5 teaching weeks of build time).
Legal laws and regulations	Must comply with the <i>Privacy Act 1988</i> — collect no more student data than needed; can't use copyrighted images for listings.
Social / ethical what's acceptable	No real money changes hands, so the app stays safe for under-18s and avoids financial-harm issues.

Common traps & misconceptions

Exam trap

1 Calling speed, security or ease 'functional' ► **X** Loses marks "A functional requirement is that SwapShelf search is fast." ✓ The fix Speed is non-functional — it's how well the search function performs, not a new feature. Functional = a behaviour you could click.

Exam trap

2 Vague, untestable wishes ► **X** Loses marks "The app must be user-friendly and good." ✓ The fix "A new student can post a listing within 2 minutes, with no instructions." A measurable target you can actually test.

Exam trap

3 A constraint dressed up as a requirement ► ✗ Loses marks “A functional requirement is that it complies with the Privacy Act.” ✓ The fix Privacy-Act compliance is a legal constraint. The requirement it drives is “messages stay private” (non-functional).

Exam trap

4 Only half a scope statement ► ✗ Loses marks “Scope: SwapShelf lists, searches and messages.” (in-scope only) ✓ The fix Name both sides: “...and out of scope: payments, delivery, ratings, native app, other schools.”

Exam trap

5 Confusing scope with a constraint ► ✗ Loses marks “The scope is that we only have 5 weeks.” ✓ The fix 5 weeks is a time constraint. Scope is what you will / won't build — e.g. “no payments in v1.”

Exam trap

6 Defining terms but ignoring the scenario ► ✗ Loses marks Textbook definitions with no SwapShelf detail in an ‘apply’ question. ✓ The fix Anchor every term to a SwapShelf feature or limit. Application is where the upper-band marks live.

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. Distinguish

Between a functional requirement and a non-functional requirement.

✗ A weak answer

"A functional requirement is what the app does, and a non-functional requirement is what it doesn't do." 'What it doesn't do' misdefines non-functional. It's not the absence of a function — it's the quality of how a function performs. Likely 0-1.

✓ A strong answer

A functional requirement is a feature or behaviour the solution must perform — *what* it does, e.g. create a listing. A non-functional requirement describes *how well* it performs a function — a quality such as speed, e.g. search results loading in under 2 seconds. Both terms correctly defined on the what / how-well axis, each with a SwapShelf example.

How the marks are awarded

- 1 mark — functional defined as a feature/behaviour (what it does).
- 1 mark — non-functional defined as a quality of performance (how well). An example each strengthens but the marks sit on the definitions.

Q2. List

SwapShelf is a school-only web app where students list, browse and message about second-hand gear to arrange free swaps.

✗ A weak answer

"Functional: the app must be fast. Non-functional: it has a search bar." Both are swapped. 'Fast' is non-functional; 'a search bar' is a functional feature. No justification given. ~0-1.

✓ A strong answer

Functional: SwapShelf must let a student create a listing — this is justified as functional because it is a behaviour the system performs that you could demonstrate by clicking.

Non-functional: search results must load in under 2 seconds — justified as non-functional because it sets *how well* the search function performs (its speed), not a new feature, and gives a measurable, testable target. Valid example of each, each with a justification tied to the what / how-well distinction.

How the marks are awarded

- 1 mark — a valid functional requirement identified.
- 1 mark — justified (a behaviour/feature the system performs).
- 1 mark — a valid non-functional requirement identified.
- 1 mark — justified (a quality of performance; measurable target rewarded).

Q3. Explain

The difference between a constraint and a requirement, using a SwapShelf example.

✗ A weak answer

"A constraint stops the app from working and a requirement is something the app needs."
Vague and partly wrong — a constraint doesn't 'stop the app working'. Misses 'imposed limit' vs 'what the solution does/quality', and gives no example. ~1.

✓ A strong answer

A constraint is a limitation imposed from outside that the team must work within — for SwapShelf, the \$0 budget forces them to use free hosting. A requirement is what the solution must do, or how well — e.g. it must let users message to arrange a swap. The constraint limits the available options; the requirement states what must be delivered. Clean contrast (imposed limit vs deliverable) with a correct SwapShelf example illustrating the difference.

How the marks are awarded

- 1 mark — constraint = an externally imposed limit/restriction.
- 1 mark — requirement = what the solution must do, or a quality it must have.
- 1 mark — a correct SwapShelf example that illustrates the difference.

Q4. Practice question

QuietHour (transfer task): a Year 12 team builds a simple focus-timer web app for the school library — one term, low budget.

✗ A weak answer

"The scope is a timer that counts down. Defining scope helps because it keeps things organised." Only an in-scope element — no out-of-scope, so the scope isn't bounded. 'Keeps things organised' doesn't link to the constraints. ~1.

✓ A strong answer

In scope: a focus timer a student can start for a chosen length, with a break alert. **Out of scope:** cross-device syncing and user accounts. Defining scope draws a clear boundary so the small team doesn't attempt more than their one-term, low-budget constraints allow — it prevents scope creep and keeps the build finishable on time. Both boundaries named; the explanation links scope directly to meeting the time/budget/team constraints.

How the marks are awarded

- 1 mark — a valid in-scope element.
- 1 mark — a valid out-of-scope element.
- 1 mark — scope explained as a boundary that prevents scope creep.
- 1 mark — links defining scope to meeting the constraints (time/budget/team size).